

Máquinas Universais em Python

Máquina de Post & Autômato de Pilha

Teoria da Computabilidade — AV2

Prof. Daniel Leal Souza · Semestre 01/2026 · Turma **CC5MA**

Augusto Pereira Rodrigues · César Ribeiro · Cauê Jidão

O que vamos mostrar

- Dois modelos distintos, dois problemas distintos:
 -  Máquina de Post (fila/FIFO) → reconhece $a^n b^n c^n$
 -  Autômato de Pilha (pilha/LIFO) → reconhece delimitadores balanceados
- Para cada um: **formalização** → **implementação** → **demonstração ao vivo**.
- Fio condutor: **mesma ideia** (controle finito + memória) → **poderes computacionais diferentes** conforme a memória.

Nada é resolvido por função pronta: o **formalismo é simulado explicitamente**.

Contexto teórico (3 min)

- **Hipótese de Church-Turing:** todos os modelos de "procedimento efetivo" têm o **mesmo poder** da Máquina de Turing.
- **Emil Post (1936):** chegou, independentemente de Turing, a um modelo equivalente baseado em **regras de produção** → a **Máquina de Post**.
- **Hierarquia de Chomsky** (o que cada memória alcança):

Memória	Modelo	Classe de linguagem
nenhuma	Autômato finito	Regular
pilha (LIFO)	Autômato de Pilha	Livre de contexto
fila (FIFO)	Máquina de Post	Recursivamente enumerável (Turing-completo)

Máquina de Post — formalização

Problema: decidir se a cadeia $\in L = \{ a^n b^n c^n \mid n \geq 1 \}$
(sensível ao contexto — **nenhum PDA reconhece**).

$M = (Q, \Sigma, \Gamma, \delta, q_0, \#, F)$

- **Q:** 10 estados (validação + cancelamento) — *> 8 estados* ✓
- $\Sigma = \{a, b, c\} \cdot \Gamma = \{a, b, c, \#\}$
- **δ :** lê a **frente** da fila, escreve no **fim** (FIFO) — 15 transições
- **Aceita** quando resta só o marcador **#**.

■ Máquina de Post — como funciona

Fase 1 — Validação estrutural (`q_val_*`)

- Garante a forma `a+b+c+` (ordem + presença), reescrevendo tudo de volta.

Fase 2 — Cancelamento cíclico (`q_skip_*`)

- Cada volta apaga 1 `a`, 1 `b` e 1 `c`.
- Repete até sobrar só `#` → **aceita**; falta de par → **rejeita**.

```
[Passo 00] q_val_a   | Fila: aabbcc#   (... passos omitidos ...)
[Passo 08] q_skip_a  | Fila: abbcc#   ← apagou um 'a'
[Passo 19] q_aceita  | Fila:           ← só restou o marcador
```

● **DEMO:** `python implementacoes/maquina_post.py` → opção 2

Autômato de Pilha — formalização

Problema: decidir se a cadeia é uma sequência **balanceada e aninhada** de $()$, $[]$, $\{\}$ (livre de contexto).

$$P = (Q, \Sigma, \Gamma, \delta, q_0, \$, F)$$

- **Q:** 9 estados (registram o tipo da última operação) — *> 8 estados* ✓
- $\Sigma = \{ (,), [,], \{, \} \}$ · $\Gamma = \{ \$, (, [, \{ \}$
- **δ :** lê (símbolo, **topo**) → empilha abertura / desempilha no fechamento
- **Aceita** quando, no fim, a pilha está reduzida ao fundo $\$$.

■ Autômato de Pilha — como funciona

- **Abrir** ([{ → **empilha** (push).
- **Fechar**)] } → topo precisa ser a abertura **do mesmo tipo** → **desempilha** (pop). Senão → **rejeita**.

```
{[()]}
```

```
[00] q_inicio    '{' [$]
```

```
[01] q_lendo... '[' [${}]
```

```
[02] q_lendo... '(' [${{}]
```

```
...
```

```
q_aceita ✓
```

```
{(())}
```

```
[00] q_inicio    '{' [$]
```

```
[01] q_lendo... '(' [${}]
```

```
[02] q_lendo... '}' [${{}]
```

```
→ ERRO: '}' não casa com topo '('
```

```
q_rejeita ✗
```

● **DEMO AO VIVO:** `python implementacoes/automato_pilha.py` → opção 2
(ou o notebook `automato_pilha.ipynb`)

Comparação dos modelos

Critério	 Máquina de Post	 Autômato de Pilha
Memória	Fila — FIFO	Pilha — LIFO
Linguagem	$a^n b^n c^n$ (sensível ao contexto)	balanceados (livre de contexto)
Poder	Turing-completo	Reconhecedor de LLC
Reconhece $a^n b^n c^n$?	Sim	Não
Aceita ϵ ?	Não ($n \geq 1$)	Sim (vazia é balanceada)

Análise técnica (3 min)

- **Por que não é trivial:** $a^n b^n c^n$ compara 3 contagens simultâneas — está além do PDA (Lema do Bombeamento p/ LLC); o balanceamento exige aninhamento por tipo — está além do autômato finito.
- **Limites:** o PDA não impõe restrições sensíveis ao contexto → por isso cada problema foi atribuído ao modelo adequado.
- **Terminação:** a Máquina de Post é um **decisor** — cada ciclo encurta a fila em 3 símbolos, logo sempre para; o PDA decide em **um passo por símbolo**.

Reprodutibilidade & entrega

- `implementacoes/` — código `.py` + notebooks visuais já **executados**
- `testes/` — **7 testes** `unittest`, todos passando
- `rastreamento/relatorio_rastreamento.md` — tabelas + traces **reais**
(`gerar_rastreamento.py` reproduz)
- `README.md` — formalização completa · `uso_ia.md` — declaração de IA

```
python -m unittest discover -s testes -v  
python rastreamento/gerar_rastreamento.py
```

GitHub: github.com/augustorodrigues-dev/M-quinas-Universais-em-Python

Obrigado!

Perguntas?

Máquina de Post & Autômato de Pilha

Augusto · César · Cauê — CC5MA — 01/2026

Estamos prontos para executar qualquer exemplo ao vivo.